

Real-Time Log Analysis Using **awk** in Linux

Objective:

By the end of this class, you will:

- To demonstrate the practical use of AWK, a powerful text-processing scripting language, in real-time scenarios such as log analysis and data filtering.
 - AWK's capabilities in pattern matching, data summarization, and automation across structured and semi-structured files.
-

Industry Scenario:

You are working as a **Junior DevOps Engineer** at a cloud-based SaaS company. The production server logs need to be monitored daily to identify:

- Error patterns
- User login statistics
- Request performance issues

Your job is to use **Linux text processing tools (awk, sed, and grep)** to extract insights from the logs and automate parts of the analysis.

Prerequisites:

- Running EC2 instance (Ubuntu)
 - Create IAM user
 - Give EC2 full permissions
 - Access EC2 via IAM use
-

Introduction to AWK

AWK is a powerful text-processing and pattern-matching tool for analyzing and manipulating text files (especially structured like logs, CSVs, etc.).

Use Cases

-  Log analysis
-  Filtering output
-  Automating tasks
-  Processing structured files (CSV, TSV, etc.)

◆ Syntax:

```
awk 'pattern { action }' filename
```

Sample File: **sample.txt**

```
john 100 dev  
jane 200 ops  
doe 150 dev
```

Basic Examples

1. ◆ Print all lines

```
awk '{ print }' sample.txt
```

Output:

```
john 100 dev
jane 200 ops
doe 150 dev
```

 *Explanation:* Prints each line of the file (default behavior).

2. ♦ Print first column (Name)

```
awk '{ print $1 }' sample.txt
```

Output:

```
john
jane
doe
```

 *Explanation:* `$1` represents the first field.

3. ♦ Print Name and Department

```
awk '{ print $1, $3 }' sample.txt
```

Output:

```
john dev
```

```
jane ops  
doe dev
```

 *Explanation:* \$1 is Name, \$3 is Department.

4. ♦ Print records where salary > 150

```
awk '$2 > 150 { print $1, $2 }' sample.txt
```

 **Output:**

```
jane 200
```

 *Explanation:* \$2 is salary; filters based on condition.

5. ♦ Change field separator (:)

```
awk -F ":" '{ print $1 }' /etc/passwd
```

 **Output:** (Example output)

```
root  
daemon  
bin
```

 *Explanation:* -F ":" tells awk to split fields using colon.

6. ♦ Add custom output separator

```
awk 'BEGIN { OFS=" | " } { print $1, $3 }' sample.txt
```

 **Output:**

```
john | dev  
jane | ops  
doe | dev
```

 *Explanation:* **OFS** sets Output Field Separator.

BEGIN & END Blocks

7. ♦ Using BEGIN and END

```
awk 'BEGIN { print "Name | Dept" } { print $1, $3 } END {  
print "Done" }' sample.txt
```

 **Output:**

```
Name | Dept  
john dev  
jane ops  
doe dev  
Done
```

 *Explanation:* **BEGIN** runs before data; **END** runs after all data.

+ Math Operations

8. ♦ Sum all salaries

```
awk '{ sum += $2 } END { print "Total:", sum }' sample.txt
```



Output:

```
Total: 450
```



Explanation: Accumulates salary from column 2.

10. ♦ Calculate average salary

```
awk '{ sum += $2; count++ } END { print "Average:",  
sum/count }' sample.txt
```



Output:

```
Average: 150
```



Explanation: Calculates average from total salary and count.

11. ♦ Filter: Salary > 120 AND Department = dev

```
awk '$3=="dev" && $2 > 120 { print $1 }' sample.txt
```

 **Output:**

```
doe
```

 *Explanation:* Multi-condition filter using `&&`.

Sample Log File: `access.log`

```
192.168.1.1 - - [11/Apr/2025:10:01:01] "GET /index.html" 200
192.168.1.2 - - [11/Apr/2025:10:01:02] "POST /login" 401
192.168.1.3 - - [11/Apr/2025:10:01:03] "GET /dashboard" 200
Malformed log line
192.168.1.4 - - [11/Apr/2025:10:01:04] "GET /settings" 500
```

Log File Analysis with AWK

12. ♦ Print line number and full content

```
awk '{ print NR, $0 }' access.log
```

 **Output:**

```
1 192.168.1.1 - - [11/Apr/2025:10:01:01] "GET /index.html"
200
2 192.168.1.2 - - [11/Apr/2025:10:01:02] "POST /login" 401
3 192.168.1.3 - - [11/Apr/2025:10:01:03] "GET /dashboard"
200
4 Malformed log line
5 192.168.1.4 - - [11/Apr/2025:10:01:04] "GET /settings" 500
```

13. ♦ Detect malformed (incomplete) lines

```
awk 'NF < 7 { print "Incomplete:", NR, $0 }' access.log
```

 **Output:**

```
Incomplete: 4 Malformed log line
```

14. ♦ Print only valid entries (7 fields)

```
awk 'NF == 7 { print "Valid Entry:", NR, $0 }' access.log
```

 **Output:**

```
Valid Entry: 1 192.168.1.1 - - [11/Apr/2025:10:01:01] "GET
/index.html" 200
Valid Entry: 2 192.168.1.2 - - [11/Apr/2025:10:01:02] "POST
/login" 401
Valid Entry: 3 192.168.1.3 - - [11/Apr/2025:10:01:03] "GET
/dashboard" 200
Valid Entry: 5 192.168.1.4 - - [11/Apr/2025:10:01:04] "GET
/settings" 500
```

15. ♦ Print only error entries (status code >= 400)

```
awk 'NF >= 7 && $NF >= 400 { print "Error:", NR, $0 }'
access.log
```

 **Output:**


```
Error: 2 192.168.1.2 - - [11/Apr/2025:10:01:02] "POST
/login" 401
Error: 5 192.168.1.4 - - [11/Apr/2025:10:01:04] "GET
/settings" 500
```

16. ♦ Print IP and endpoint for success requests (200 OK)

```
awk 'NF == 7 && $NF == 200 { print "Line:", NR, "IP:", $1,
"Endpoint:", $6 }' access.log
```

Output:

```
Line: 1 IP: 192.168.1.1 Endpoint: "GET
Line: 3 IP: 192.168.1.3 Endpoint: "GET
```

17. ♦ Clean output: Show Method & Resource for success

```
awk 'NF == 7 && $NF == 200 {
    gsub("/"/, "", $6);
    gsub("/"/, "", $7);
    print "Line:", NR, "IP:", $1, "Method:", $6, "Resource:",
$7
}' access.log
```

Output:

```
Line: 1 IP: 192.168.1.1 Method: GET Resource: /index.html
Line: 3 IP: 192.168.1.3 Method: GET Resource: /dashboard
```



Explanation: `gsub()` removes double quotes.



Summary:

This class explored the capabilities of **AWK**, a powerful scripting language used for text processing, especially in UNIX/Linux environments. Through practical examples, we demonstrated how AWK can:

- Extract specific columns from structured data (`sample.txt`)
- Filter rows based on conditions (e.g., salaries above a threshold)
- Format and summarize data (e.g., totals, averages, and counts)
- Process and validate web server logs (`access.log`)
- Identify malformed entries and extract meaningful patterns

We also utilized advanced AWK features like custom field separators, `BEGIN` and `END` blocks, and associative arrays for grouping and aggregating data.